

Full Length Article

Dual object graph and bisimulation metric for object-goal navigation in unfamiliar environment

Yiyue Meng^{a,b}, Chi Guo^{a,c,*}, Aolin Li^a, Kang Zhou^d^a GNSS Research Center, Wuhan University, Wuhan, 430079, China^b Electronic Information School, Wuhan University, Wuhan, 430079, China^c Hubei LuoJia Laboratory, Wuhan, 430079, China^d Department of Computer Science, City University of Hong Kong, Hong Kong, 999077, China

ARTICLE INFO

Keywords:

Object-goal navigation

Object graph

Bisimulation metric

Reinforcement learning

ABSTRACT

Object-goal navigation aims to guide an agent to find a specific target object in an unfamiliar environment based on first-person visual observations. It requires the agent to learn informative visual representations and robust navigation policy. To promote these two components, we propose two complementary techniques, dual object graph (DOG) and bisimulation metric (BM). DOG integrates current and historical object relationships, including category proximity and spatial correlation. It constructs a current object graph (COG) to model real-time object relationships and maintains a historical object graph (HOG) to preserve long-term object relationships. DOG improves visual representation learning. Both DOG and BM aim to improve robust navigation policy, enabling the agent to escape from deadlock states, such as looping or getting stuck. Specifically, BM is a self-supervised reinforcement learning (RL) technique that groups behaviorally similar observations in representation space. In the process, BM learns robust latent representations that capture only the task-relevant information from observations against distractions such as variations in background or viewpoint, thus providing guidance for effective navigation policy. Experiments in the AI2-Thor and RoboThor environment demonstrate that our method significantly improves the effectiveness and efficiency of navigation in unfamiliar environments, and real-world deployment further validates its transferability.

1. Introduction

Object-goal navigation aims to guide an agent to find a given target object in an unfamiliar environment using only first-person visual observations (Li et al., 2022; Sun et al., 2024; Zhang et al., 2022). This requires the agent not only to recognize objects, like what a remote control looks like, but also to understand where it might be placed. Similar to human navigation in unfamiliar environments, the agent must memorize object placements to construct an object graph that facilitates finding the target object. For example, “to find the remote control,” the agent may first find the relatively large television based on object relationships in the object graph, such as “the remote control is next to the television.” Searching around the television can help the agent to find the remote control. However, due to the complexity of the placements, the agent may also fail to find the target object and get stuck. Thus, learning informative visual representations containing the object graph and effective navigation policy are two crucial components.

Most previous works have proposed graph-based and meta-learning (ML)-based methods to improve the two components mentioned above.

For the first component, works (Du et al., 2020; Yang et al., 2018; Zhang et al., 2021) construct object graphs to enrich visual representations, such as “the alarm clock is next to the bed.” They focus on encoding various relationships between objects. Yang et al. (2018) obtains prior relationships from the Visual Genome (VG) dataset (Krishna et al., 2017). Du et al. (2020) learns co-occurrence relationships using Faster R-CNN (Ren et al., 2015). Zhang et al. (2021) proposes a zone-wise graph to cluster object relationships with specific zones. However, these works only construct graphs from current observations while neglecting to preserve long-term graphs, which is crucial for long-distance navigation. Besides, they use the graph convolutional network (GCN) (Kipf & Welling, 2016) to encode object relationships, treating all objects equally and discarding the agent’s attention. For example, to “find the laptop,” the agent may focus on the closer desk while ignoring the distant kettle that has a low correlation with the laptop. Thus, although these methods use object graphs to learn informative visual representations, their generalization ability in unfamiliar environments remains limited.

For the second component, works (Yang et al., 2018; Zhu et al., 2017) use unsupervised reinforcement learning (RL) algorithms to train

* Corresponding author.

E-mail addresses: mengyiyue@whu.edu.cn (Y. Meng), guochi@whu.edu.cn (C. Guo), 2020302142188@whu.edu.cn (A. Li), kangzhou@cityu.edu.hk (K. Zhou).<https://doi.org/10.1016/j.neunet.2026.108921>

Received 6 November 2024; Received in revised form 29 October 2025; Accepted 30 March 2026

Available online 6 April 2026

0893-6080/© 2026 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

navigation policy. However, when navigating in new environments that differ from the training data, RL often struggles with adaptation problems. To address this, works (Wortsman et al., 2019; Zhou et al., 2023) propose using ML to adapt the policy when the environment changes. They demonstrate a more robust navigation policy. However, ML requires adequate high-quality data and additional hyperparameter tuning.

Due to the various object categories and different environments, manually labeling object relationships from various observations is difficult. We propose a current object graph (COG) to learn real-time object relationships, including category proximity and spatial correlation. COG uses Faster R-CNN to detect visible objects from current observations and establish co-occurrence relationships. Besides, in long-distance navigation, we propose the “relationship forgetting problem,” where the agent may lose information about previously encountered object relationships. For example, to “find a cellphone in a large living room,” the agent observes a table near a sofa. If the agent encounters this setup again, it should recognize that it has already navigated through this area or a similar layout, thus avoiding unnecessary exploration. Therefore, we propose a historical object graph (HOG) to preserve long-term object relationships. HOG retains sequential encountered objects over time using a long short-term memory (LSTM) (Hochreiter & Schmidhuber, 1997) network and establishes relationships based on prior knowledge of environments. COG and HOG together form our proposed dual object graph (DOG). To help the agent focus on valuable relationships, we apply the graph attention network (GAT) (Veličković et al., 2017) to learn both COG and HOG. Using DOG, the agent can narrow down the search area, thus improving the effectiveness and efficiency of navigation.

Due to the complexity of the environment, the agent may get stuck, such as repeatedly performing the same rotation action. Relying solely on RL for training cannot resolve this problem. Inspired by human learning-to-learn, ML learns good parameter initialization with only a few gradient updates and adjusts the policy parameters as the environment changes. While ML is effective, it requires a sufficiently high-quality support set for meta-training to avoid overfitting. Besides, we must tune the hyperparameters to ensure early stopping, which is always challenging. We propose a bisimulation metric (BM) to guide the agent to learn only the task-relevant policy. BM is a self-supervised RL technique that quantifies behavioral similarity between observations. The training data for BM includes observations, actions, and rewards, which can be directly obtained throughout the RL training process. BM learns robust latent representations that encode only task-relevant information from observations against distractions, thus guiding the agent to take effective actions. For example, when navigating through a corridor, objects such as windows on either side are distractions that might mislead the agent and cause it to get stuck. Finally, we combine the standard asynchronous advantage actor-critic (A3C) RL algorithm (Mnih et al., 2016) with BM to train navigation policy. Our method is principled and modular, allowing it to be combined with other navigation methods. Experiments show that the combination of DOG and BM significantly improves the effectiveness and efficiency of navigation in unfamiliar environments. The main contributions of this paper are summarized as follows:

- We propose DOG for indoor environments. DOG constructs a COG to model real-time object relationships and maintains a HOG to preserve long-term object relationships. We use GAT to encode valuable relationships from DOG, which can narrow down the search area.
- We propose BM to improve navigation policy. BM encodes only task-relevant information from observations against distractions, guiding the agent to take effective actions. We combine DOG and BM to train a robust navigation policy.
- Experiments demonstrate that our proposed method significantly improves the effectiveness and efficiency of navigation in unfamiliar environments. Furthermore, we validate the practical applicability of our method through successful deployment on a real-world robot.

2. Related work

2.1. Object-goal navigation

Object-goal navigation, a fundamental task in robotics and artificial intelligence, has attracted increasing attention recently. Traditional methods often construct environment maps and divide the navigation task into localization, mapping, and path planning (Oriolo et al., 1995). Dissanayake et al. (2001) proposes the simultaneous localization and mapping (SLAM) technique, which uses maps to guide the agent towards the goal. However, these maps are often unavailable in unfamiliar environments.

Recently, RL has been applied to design end-to-end navigation methods. In unfamiliar environments without maps, RL enables the agent to map visual observations directly to navigation policy. Zhu et al. (2017) uses RL to search for a target object, producing navigation actions based on visual observations. Wortsman et al. (2019) proposes an adaptive navigation method that combines RL and ML to adjust policy in new environments. Mousavian et al. (2019) integrates semantic segmentation and detection masks into visual representations and uses RL to learn navigation policy. Druon et al. (2020) proposes an intermediate representation called context grid and uses RL to predict actions. Mayo et al. (2021) designs an attention probability model to encode object semantic and spatial information and uses RL to improve the agent’s policy. Du et al. (2021) proposes a visual transformer to learn informative visual representations and uses RL to map them to navigation actions. Du et al. (2023) estimates navigation states effectively by exploring relationships among historical navigation states and uses RL to learn navigation policy. However, these methods lack object graphs, limiting their generalization performance in unfamiliar environments.

Therefore, Yang et al. (2018) uses GCN to exploit object relationships obtained from the external knowledge base. Du et al. (2020) uses GCN to learn co-occurrence relationships, including category closeness and spatial correlations. Zhang et al. (2021) proposes a hierarchical object-to-zone graph and uses GCN to exploit object relationships within specific zones. Pal et al. (2021) proposes a hierarchical object relationship learning approach to the object-goal navigation problem by understanding the role of semantic context. Dang et al. (2023b) proposes the unbiased directed object attention (DOA) graph method, and Dang et al. (2023a) extends it by integrating a meta-ability reward mechanism. Xu et al. (2024) proposes to align knowledge graphs with visual perception and uses GCN to extract object relationships. However, these methods may suffer from the “relationship forgetting problem,” where the agent may lose information about previously encountered object relationships. Besides, using GCNs may limit the agent’s attention to important relationships. In contrast, our method exploits both real-time and long-term object relationships by constructing COG and HOG. We use GAT to help the agent focus on valuable relationships.

2.2. Representation learning

Recently, representation learning has made significant progress, focusing on encoding useful information from high-dimensional observations. For example, van den et al. (2018) proposes contrastive predictive coding (CPC), which uses a contrastive objective to predict future latent states. Mazouze et al. (2020) extends this by adding a mutual-information maximization auxiliary loss that encourages representations capture predictive information about future states. Building on these works, bisimulation-based methods explicitly align latent distances with behavioral similarities. Zhang et al. (2020) employs a bisimulation metric to learn task-relevant representations from images without reconstruction, demonstrating robustness to visual distractors. More recently, Wang et al. (2023) uses an inverse-dynamic bisimulation metric to guide exploration, while Zhang et al. (2025) revisits bisimulation representations with improved update rules. Besides, Yarats et al. (2021) broadens the scope by leveraging prototypical

latent structures for RL. Grill et al. (2020) demonstrates how strong representations can be learned without contrastive negative samples. These works highlight how representation learning is shifting from purely visual or reconstruction-based frameworks toward behaviorally grounded, task-relevant metrics. Our method follows this paradigm by using BM to guide the agent to learn only the task-relevant policy, thus helping the agent to escape from deadlock states.

3. Proposed method

Our objective is to acquire informative visual representations and effective navigation policy for object-goal navigation. To achieve this, our navigation framework comprises two major components, as illustrated in Fig. 1: (1) learning visual representations from RGB observations. In this component, we introduce DOG to extract informative visual representations containing both real-time and long-term object relationships. (2) learning effective navigation policy. To prevent the agent from being trapped in deadlock states, we propose BM to encode only task-relevant information from our visual representations, allowing the agent to take effective actions through RL.

3.1. Task definition

Object-goal navigation requires finding a specified target object in a given environment. During navigation, the agent perceives only available RGB images from an egocentric viewpoint and predicts its next actions. The agent does not have access to information about the entire

environment, such as a map. The environment is divided into grids, each representing a distinct location. The available actions include *MoveAhead*, *RotateLeft*, *RotateRight*, *LookUp*, *LookDown*, and *Done*. *MoveAhead* action moves the agent forward by 0.25 m. *RotateLeft* and *RotateRight* rotate the agent by 45°. *LookUp* and *LookDown* tilt the agent's view upward and downward by 30°.

A successful navigation episode is defined by the following conditions: (1) the agent selects the termination action *Done* within a specified maximum number of steps. (2) the target object is within the agent's current field of view. (3) the distance between the agent and the target object is less than the visibility threshold (i.e., 1.5 m). If these conditions are not met, the episode concludes as a failure.

At the beginning of each navigation episode, the agent is assigned a random target object g from the object set like $\{\text{AlarmClock, Book, ..., Toaster}\}$ and is initially placed at a random location $l = \{x, z, \theta_{yaw}, \theta_{pitch}\}$ in a random room, where x and z represent the plane coordinates, and θ_{yaw} and θ_{pitch} represent the yaw and pitch angles of the agent. At each time step t , the agent captures the current observation o_t and learns the optimal action policy $\pi(a_t|o_t, g)$, where a_t is selected from the available discrete actions mentioned above. The agent will continue taking actions until the terminal action *Done* is executed.

3.2. Dual object graph

Since the agent can only perceive an egocentric view of the environment, learning informative visual representations is crucial. Inspired by human navigation, we aim to fully exploit both short-term and

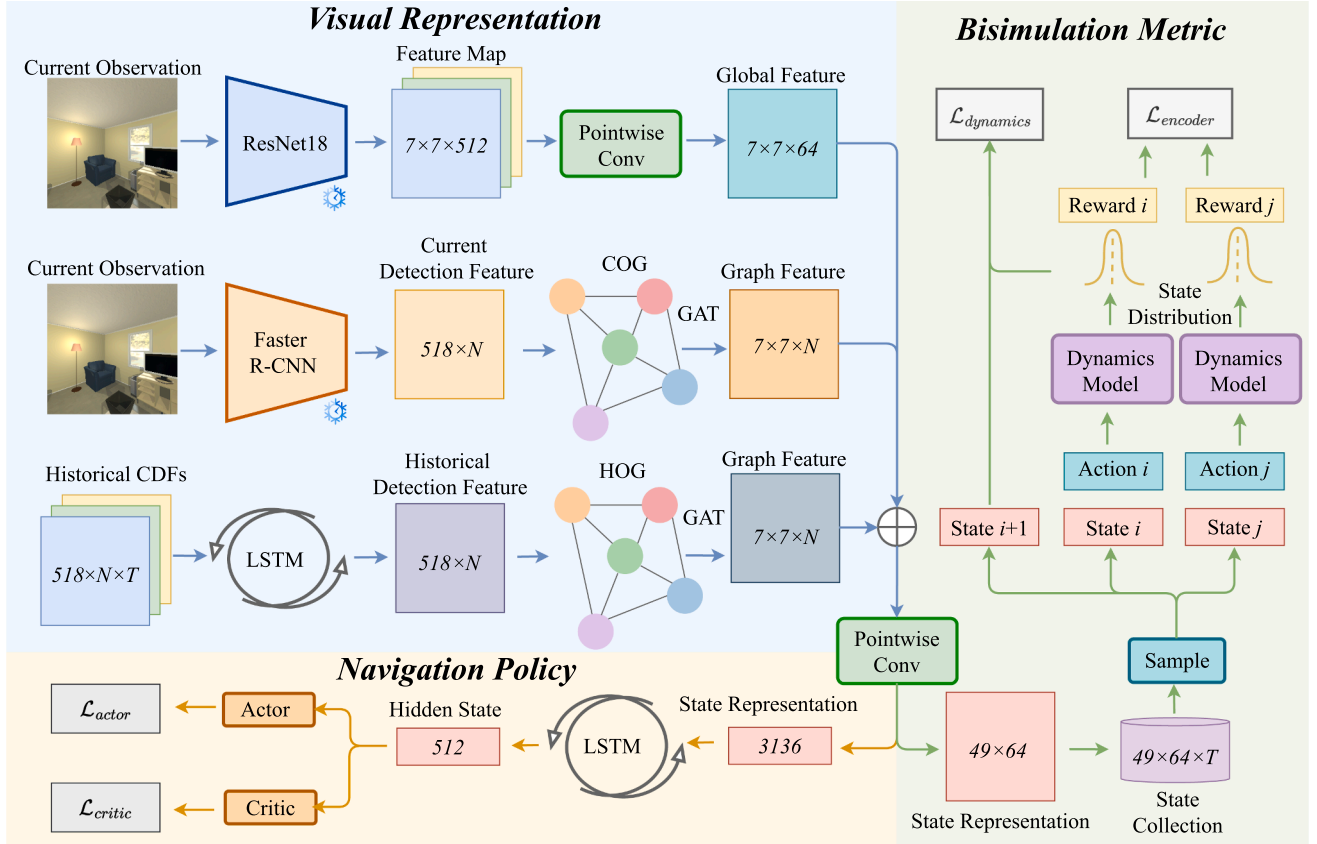


Fig. 1. Overview of our proposed navigation framework. Our framework comprises three parts: visual representation, bisimulation metric (BM), and navigation policy. The visual representation combines a global feature extracted from ResNet18 and two graph features from COG and HOG using GAT. COG exploits object relationships from current observations. HOG preserves long-term object relationships over a navigation episode of T time steps. BM is a self-supervised RL technique that learns only task-relevant information from the visual representation. The input of BM is visual representations, actions, and rewards over a navigation episode. Our framework combines BM with A3C to learn effective navigation policy. During training, our framework generates dynamics loss and encoder loss from BM, and actor loss and critic loss from A3C. These losses are backpropagated through the entire framework to update parameters.

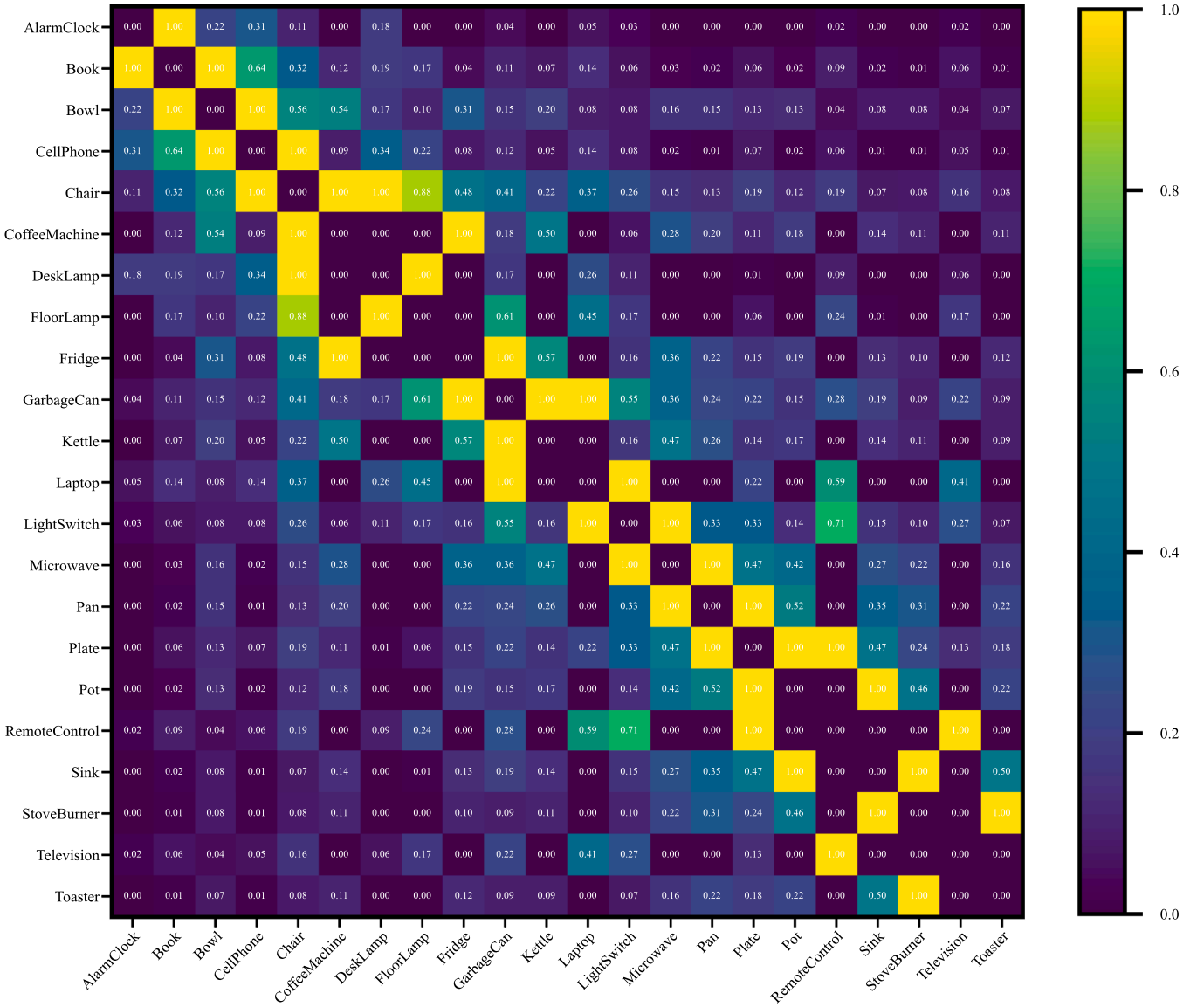


Fig. 2. The visualization of co-occurrence frequency. We select 22 target objects from the AI2-Thor environment to represent: *AlarmClock*, *Book*, *Bowl*, *CellPhone*, *Chair*, *CoffeeMachine*, *DeskLamp*, *FloorLamp*, *Fridge*, *GarbageCan*, *Kettle*, *Laptop*, *LightSwitch*, *Microwave*, *Pan*, *Plate*, *Pot*, *RemoteControl*, *Sink*, *StoveBurner*, *Television*, *Toaster*.

long-term object relationships, including category proximity and spatial correlation. To achieve this, we introduce a dual object graph (DOG), combining the current object graph (COG) and historical object graph (HOG) to help the agent learn informative visual representations. Besides, we use the GAT to focus on valuable relationships that help narrow down the search area.

Current object graph. To learn real-time object relationships at each time step, we first need to find all the objects in an image. To achieve this, we train an object detector, i.e., Faster R-CNN (Ren et al., 2015), to identify all the visible objects of interest. If there are multiple instances of an object category, we only select the one with the highest confidence score. Faster R-CNN transforms N encoded d -dimension features $\mathbb{R}^{d \times N}$ (i.e., $d = 512$) into N detection results, including bounding boxes and confidence scores. We concatenate the d -dimension feature $\mathbb{R}^{d \times N}$, bounding box $\mathbb{R}^{4 \times N}$, and confidence score $\mathbb{R}^{1 \times N}$ to create the current detection feature (CDF). If certain objects are not present in the input image, their bounding boxes and confidence scores are recorded as 0. To incorporate information about the target object, a one-hot encoded target vector $\mathbb{R}^{1 \times N}$ is also concatenated, resulting in the current node

feature (CNF) of size $\mathbb{R}^{518 \times N}$. Each node feature represents a vertex, enabling the construction of COG. COG is denoted as $G_{cog}(V, E)$, where V is nodes and E is the edges between nodes. The edges E are established based on object co-occurrence. Specifically, if two objects are detected in one image, an edge is established between them. Besides, each object node is assigned a self-loop edge.

Historical object graph. To address the “relationship forgetting problem,” we maintain HOG that preserves long-term object relationships. To achieve this, we retain sequentially encountered CDFs over a navigation episode of T time steps using an LSTM network. The hidden states produced by the LSTM create the historical detection feature (HDF) of size $\mathbb{R}^{518 \times N}$, enabling the construction of the historical node feature (HNF) of HOG. HOG is denoted as $G_{hog}(V, E)$, where V is nodes and E is the edges between nodes. To establish the edges E , we apply a random walk approach in the environment to get 10,000,000 more prior co-occurrence frequencies among 108 object categories. As illustrated in Fig. 2, the attribute of the edge between objects is represented by a “weight.” The “weight” is calculated based on normalized co-occurrence frequency. For example, the weight of the edge between *AlarmClock*

and *DeskLamp* is calculated by normalizing its frequency against the frequency of *DeskLamp*. As we are only interested in the most probable edges between objects, we prune the edges with a “weight” less than a threshold θ_{th} (i.e., $\theta_{th} = 0.1$). Besides, each object node is assigned a self-loop edge.

Object graph representation. We introduce the graph attention network (GAT) to learn object graph representations for COG and HOG. Compared to GCN, GAT can help the agent focus on valuable relationships and thus narrow down the search area. We illustrate the process as follows:

$$v_i^{l+1} = \sigma \left(\sum_{j \in N(i)} \alpha_{ij}^{(l)} W^{(l)} v_j^{(l)} \right) \quad (1)$$

$$\alpha_{ij}^{(l)} = \frac{\exp(\text{LeakyReLU}(a^{(l)}[W^{(l)} v_i^{(l)} || W^{(l)} v_j^{(l)}]))}{\sum_{k \in N(i)} \exp(\text{LeakyReLU}(a^{(l)}[W^{(l)} v_i^{(l)} || W^{(l)} v_k^{(l)}]))} \quad (2)$$

where $v_i^{(l)}$ is the feature of node i at the l -th layer, $v_i^{(l+1)}$ is the updated node feature at the $l+1$ th layer, $N(i)$ is the neighbors of node i , σ is the ELU activation function, $W^{(l)}$ is the weight matrix at the l th layer, $\alpha_{ij}^{(l)}$ is the attention coefficient between nodes i and j , and $[W^{(l)} v_i^{(l)} || W^{(l)} v_j^{(l)}]$ is the concatenation of nodes i and j . We use K (i.e., $K = 4$) attention heads to capture independent object relationships from various perspectives, encoding informative visual representations that incorporate prior knowledge of category proximity and spatial correlation.

3.3. Visual representation

Although we introduce DOG to learn object graph representations, the agent needs a global feature to capture a panoramic view of the environment. Similar to Wortsman et al. (2019), we use ResNet18 (He et al., 2016) pre-trained on the ImageNet dataset (Deng et al., 2009) to extract the global visual feature of the current observation. Specifically, we remove the last fully connected layer and average pooling layer of ResNet18 to obtain a feature map of size $\mathbb{R}^{7 \times 7 \times 512}$. We then apply a pointwise convolution layer with ReLU activation to reduce the number of channels from 512 to 64, producing the visual feature of size $\mathbb{R}^{7 \times 7 \times 64}$ while preserving the spatial dimensions. This feature is concatenated with object graph representations to form our final visual representations.

3.4. Bisimulation metric

Due to the complexity of the environment, the agent may be trapped in deadlock states. We argue that representing all elements observed in the environment, whether relevant to the task or not, can distract the navigation policy. To address this, we propose BM to group behaviorally similar observations in latent representation space. BM encodes only task-relevant information from our visual representations, allowing the agent to take effective actions from RL.

Task-relevance definition. Object-goal navigation is defined as a Markov decision process (MDP) represented by the tuple $\mathcal{T} = (S, \mathcal{A}, \mathcal{P}, R, \gamma)$, where S is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}(s'|s, a)$ is the probability of transitioning from state s to state s' , and $\gamma \in [0, 1)$ is the discount factor. The agent selects actions $a \in \mathcal{A}$ according to a policy $a \sim \pi(s)$, which updates the state $s' \sim \mathcal{P}(s, a)$, yielding a reward $r = R(s)$. Since the objective of the RL-based agent is to maximize the cumulative discounted rewards, we can use the reward signal r to help determine task-relevance, as shown by Jonschkowski and Brock (2015). Besides, state elements are relevant not only if they influence the current reward, but also if they influence state elements in the future that in turn influence future rewards. Thus, the state distribution $\mathcal{P}$ can also help determine task-relevance.

Bisimulation relation. To learn these task-relevant elements from observations, various forms of state abstractions have been defined in MDP to group states into clusters, thus preserving some property such

as task-relevance (Li et al., 2006). A bisimulation is a form of state abstraction that clusters states s_i and s_j that are considered “behaviorally equivalent” (Li et al., 2006). Based on the above analysis, states s_i and s_j have bisimulation relation if they share both the same immediate reward and equivalent distribution over the next states (Givan et al., 2003; Larsen & Skou, 1989):

$$R(s_i, a) = R(s_j, a), \forall a \in \mathcal{A} \quad (3)$$

$$\mathcal{P}(G|s_i, a) = \mathcal{P}(G|s_j, a), \forall a \in \mathcal{A}, \forall G \in S_B \quad (4)$$

where S_B is the partition of S under the bisimulation relation B , G is the set of all groups of equivalent states, and $\mathcal{P}(G|s_j, a) = \sum_{s' \in G} \mathcal{P}(s'|s_j, a)$.

Bisimulation metric. To extract S_B , BM (Castro, 2020; Ferns et al., 2011; Ferns & Precup, 2014) softens the concept of state partitions and defines a pseudometric space $(S, d) : S \times S \mapsto \mathbb{R}_{\geq 0}$, where d is a distance function that quantifies both a distance between rewards (to soften Eq. (3)) and a distance between state distributions (to soften Eq. (4)). If d is zero, then states s_i and s_j are considered “behaviorally equivalent.” Inspired by Zhang et al. (2020), we introduce their BM to train an encoder $\phi : S \mapsto \mathcal{Z}$ that captures state representations (i.e., our visual representations) that are suitable for object-goal navigation while discarding any information that is irrelevant for navigation policy. BM softens the d between states towards l_1 distance in the representation space of the encoder ϕ , denoted as $d(s_i, s_j) := ||\phi(s_i) - \phi(s_j)||_1$. As BM is a self-supervised RL technique, we directly draw batches of state pairs from an episode of the navigation and minimize the mean square error between BM and l_1 distance in the latent space:

$$\mathcal{L}(\phi) = (||z_i - z_j||_1 - |r_i - r_j| - \gamma W_2(\hat{\mathcal{P}}(\cdot|z_i, a_i), \hat{\mathcal{P}}(\cdot|z_j, a_j)))^2 \quad (5)$$

where $z_i = \phi(s_i)$ and $z_j = \phi(s_j)$ are the encoded features of states s_i and s_j , r is rewards, $\bar{z}$ is $\phi(s)$ with stop gradients, and γ is the discount factor. Furthermore, Eq. (5) uses a probabilistic dynamics model $\hat{\mathcal{P}}$ that outputs a Gaussian distribution. For this reason, we use the 2-Wasserstein metric W_2 (Villani et al., 2009) to measure the distance between two state distributions since the W_2 metric has a convenient closed form:

$$W_2(\mathcal{N}(\mu_i, \Sigma_i), \mathcal{N}(\mu_j, \Sigma_j))^2 = ||\mu_i - \mu_j||_2^2 + ||\Sigma_i^{1/2} - \Sigma_j^{1/2}||_F^2 \quad (6)$$

where $||\cdot||_F$ is the Frobenius norm. We continue to use the l_1 norm for all other distances.

Bisimulation metric for RL. We combine BM and RL to extract task-relevant information from state representations (i.e., our visual representations), which enables the agent to learn effective navigation policy. We train BM by iteratively updating the encoder ϕ and the dynamics model $\hat{\mathcal{P}}$. As shown in Fig. 1, our visual representations are the concatenation of global visual features $f_{global} \in \mathbb{R}^{7 \times 7 \times 64}$ and object graph representations (i.e., COG feature $f_{cog} \in \mathbb{R}^{7 \times 7 \times N}$ and HOG feature $f_{hog} \in \mathbb{R}^{7 \times 7 \times N}$ obtained by GAT). The encoder ϕ is a pointwise convolution layer and produces the state representation $s_i \in \mathbb{R}^{7 \times 7 \times 64}$. To design the dynamics model $\hat{\mathcal{P}}$ that predicts the distribution of the next state s_{i+1} , we use a multi-head attention mechanism (Vaswani et al., 2017) to predict the next state based on the current state $s_i \in \mathbb{R}^{7 \times 7 \times 64}$ and the current action $a_i \in \mathbb{R}^{1 \times 6}$, as illustrated in Fig. 3. Specifically, we reshape the state representation into $s_i \in \mathbb{R}^{49 \times 64}$, pass the action $a_i \in \mathbb{R}^{1 \times 6}$ through a full connection layer, and replicate it to produce a new action feature $a_i \in \mathbb{R}^{49 \times 64}$. $\hat{\mathcal{P}}$ is formulated as follows:

$$Q = a_i W_q, K = s_i W_k, V = s_i W_v \quad (7)$$

$$\text{head}_j(Q, K, V) = \text{softmax} \left(\frac{Q_j K_j^T}{\sqrt{d}} \right) V_j \quad (8)$$

$$O = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_o \quad (9)$$

$$\mu = \text{Linear}(O), \Sigma = \text{Linear}(O) \quad (10)$$

where Q , K , and V are the query, key, and value, obtained by weight matrices W_q , W_k , and W_v . These matrices are divided into h heads (i.e., $h = 4$), with each head independently computing scaled dot-product attention, and d (i.e., $d = 64$) is the dimension. After concatenating the

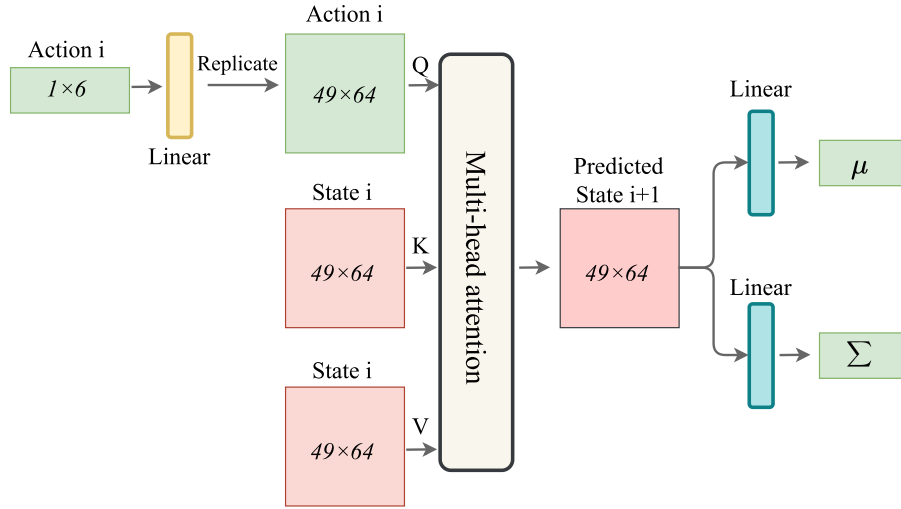


Fig. 3. The illustration of the dynamics model. We use action $a \in \mathbb{R}^{49 \times 64}$ as the query and state $s \in \mathbb{R}^{49 \times 64}$ as the key and value. We use the multi-head attention mechanism to predict the next state's distribution, including mean μ and variance Σ .

output of each head, we linearly transform it using weight matrix W_o to obtain the final output O . Based on this output, we use two fully connected layers to get the mean μ and variance Σ of the next state distribution in Eq. (6), respectively. We design the dynamics loss using a maximum likelihood loss for Gaussian distribution to measure the error between the predicted next state and true s_{i+1} (Zhang et al., 2020):

$$\mathcal{L}_{dynamics} = \frac{(s_{i+1} - \mu)^2}{2\Sigma} + \log \Sigma^{1/2} \quad (11)$$

We combine BM with RL to train effective navigation policy. As outlined in Algorithm 1, the policy π guides the agent's actions in the environment. The state collection C is obtained from each training episode and organized into a batch B for training. BM produces two losses, including dynamics loss and encoder loss. This combined loss will back-propagate through the network to update the state representation.

Algorithm 1 Incorporating bisimulation metric into RL

- 1: **for** time step $t = 0$ to T in an episode **do**
 - 2: Input global visual feature f_{global}
 - 3: Input COG feature f_{cog}
 - 4: Input HOG feature f_{hog}
 - 5: Make state $s_t = \text{concat}(f_{global}, f_{cog}, f_{hog})$
 - 6: Encode state $z_t = \phi(s_t)$
 - 7: Select action $a_t \sim \pi(z_t)$
 - 8: Record data $C \leftarrow C \cup \{s_t, a_t, s_{t+1}, r_t\}$
 - 9: **end for**
 - 10: Make batch $B_i = C$
 - 11: Permute batch $B_j = \text{permute}(B_i)$
 - 12: Train encoder $\mathcal{L}_{encoder} = \mathbb{E}_{B_i, B_j}[\mathcal{L}(\phi)]$
 - 13: Train dynamics $\mathcal{L}_{dynamics} = \mathbb{E}[(\hat{P}(\phi(s_t), a_t) - \bar{z}_{t+1})^2]$
-

3.5. Navigation policy

We adopt the standard asynchronous advantage actor-critic (A3C) algorithm to train navigation policy. Our visual representations are fed into an LSTM network. The hidden states of the LSTM are then fed into the A3C model. A3C produces two outputs, including policy and value. The agent samples the action from the predicted policy with the highest probability, and the output value is used to train the policy. The actor loss and critic loss are formulated as:

$$\mathcal{L}_{actor} = -\mathbb{E}[\log \pi(a_t | s_t; \theta) \cdot (\sum_{t=0}^T \gamma^t r_t - V(s_t; \theta_v))] \quad (12)$$

$$\mathcal{L}_{critic} = \mathbb{E}[(\sum_{t=0}^T \gamma^t r_t - V(s_t; \theta_v))^2] \quad (13)$$

where $\pi(a_t | s_t; \theta)$ is the probability of taking the action a_t given the state s_t under the policy network π parameterized by θ . $\sum_{t=0}^T \gamma^t r_t - V(s_t; \theta_v)$ is the advantage at time step t , which defines the difference between the cumulative discounted rewards and the estimated value, $\sum_{t=0}^T \gamma^t r_t$ is the cumulative discounted rewards, and $V(s_t; \theta_v)$ is the value of the state s_t estimated by the critic network parameterized by θ_v .

After combining BM and RL, the total loss function of our proposed object-goal navigation network is defined as the weighted sum of individual losses, including actor loss, critic loss, encoder loss, and dynamics loss. Subsequently, this combined loss is backpropagated through the network to update its parameters. Specifically, the total loss $\mathcal{L}_{total}$ is given by:

$$\mathcal{L}_{total} = \lambda_a \mathcal{L}_{actor} + \lambda_c \mathcal{L}_{critic} + \lambda_e \mathcal{L}_{encoder} + \lambda_d \mathcal{L}_{dynamics} \quad (14)$$

where the weights λ_a , λ_c , λ_e , and λ_d correspond to actor loss, critic loss, encoder loss, and dynamic loss, respectively. In our experiments, we specify $\lambda_{actor} = 1$, $\lambda_{critic} = 0.5$, $\lambda_{encoder} = 0.5$, and $\lambda_{dynamic} = 1$. The optimization of the total loss function is aimed at enhancing the overall performance of the A3C algorithm for training effective navigation policy.

4. Experiments

4.1. Datasets

We evaluate the generalization ability of our proposed object-goal navigation method in unfamiliar environments using the AI2-Thor (Kolve et al., 2017) and RoboThor (Deitke et al., 2020) environments, which provide near photo-realistic observations in 3D indoor scenes. The AI2-Thor environment contains 120 scenes in four types: kitchen, living room, bedroom, and bathroom. Each room is unique in terms of furniture placement and the types of objects present. Following prior studies (Du et al., 2020, 2021; Xu et al., 2024; Zhang et al., 2021), we select the first 20 scenes of each scene type as training scenes, the subsequent 5 scenes as validation scenes, and the last 5 scenes as test scenes to perform object-goal navigation in unfamiliar environments. We select 22 types of objects as targets, ensuring that each scene type contains at least 4 objects. The target objects include AlarmClock, Book, Bowl, CellPhone, Chair, CoffeeMachine, DeskLamp, FloorLamp, Fridge, GarbageCan, Kettle, Laptop, LightSwitch, Microwave, Pan, Plate, Pot, RemoteControl, Sink, StoveBurner, Television, and Toaster.

Table 1
Comparisons of existing navigation frameworks and our proposed framework.

Method	Target Object	Observation	Knowledge Graph	Navigation Policy
Baseline (Zhu et al., 2017) (2017 ICRA)	Glove	ResNet50	None	A3C
SP (Yang et al., 2018) (2019 ICLR)	Glove	ResNet50	VG + GCN	A3C
SAVN (Wortsman et al., 2019) (2019 CVPR)	Glove	ResNet18	None	A3C + ML
CG (Druon et al., 2020) (2020 RAL)	Word2Vec	ResNet50	None	A3C
ORG (Du et al., 2020) (2020 ECCV)	One-Hot	ResNet18	ORG + GCN	A3C
ORG + TPN (Du et al., 2020) (2020 ECCV)	One-Hot	ResNet18	ORG + GCN	A3C + TPN
MJOLNIR (Pal et al., 2021) (2021 CoRL)	Glove	ResNet18	Context Matrix + GCN	A3C
VTNet (Du et al., 2021) (2021 ICLR)	One-Hot	ResNet18	None	A3C
HOZ (Zhang et al., 2021) (2021 ICCV)	One-Hot	ResNet18	HOZ Graph + GCN	A3C
HiNL (Du et al., 2023) (2023 CVPR)	One-Hot	ResNet18	None	A3C
DAT (Dang et al., 2023b) (2023 ICCV)	One-Hot	ResNet18	DOA	A3C
MT (Dang et al., 2023a) (2023 ICML)	One-Hot	ResNet18	DOA	A3C + MA
AKGVP (Xu et al., 2024) (2024 ICRA)	One-Hot	ResNet18	CLIP + GCN	A3C
Ours	One-Hot	ResNet18	COG + HOG + GAT	A3C + BM

The RoboThor environment contains 89 apartments, 75 for training and validation, which has a 2.4 times larger area and a 5.5 times longer trajectory length than AI2-Thor. Following prior studies (Dang et al., 2023a,b; Zhang et al., 2021), we select 60 apartments for training, 5 for validation, and 10 for testing. We select 12 types of objects as targets, including *AlarmClock*, *Book*, *Bowl*, *CellPhone*, *Chair*, *DeskLamp*, *FloorLamp*, *GarbageCan*, *Laptop*, *Plate*, *Pot*, *Television*.

4.2. Metrics

Following prior studies (Du et al., 2020, 2021; Xu et al., 2024; Zhang et al., 2021), we evaluate the generalization ability of our proposed method in unfamiliar environments using the success rate (SR), the success weighted by path length (SPL), and the success weighted by action efficiency (SAE). SR is the agent's success rate in finding the target object, calculated as $SR = \frac{1}{N} \sum_{i=1}^N S_i$, where N is the total number of episodes, and S_i is a binary function indicating whether the i th episode is successful. SPL considers both success rate and path length, calculated as $SPL = \frac{1}{N} \sum_{i=1}^N S_i \frac{L_i}{\max(L_i, P_i)}$, where L_i is the optimal trajectory length of the i th episode, and P_i is the path length of the i th episode. SAE measures the efficiency of all actions, calculated as $SAE = \frac{1}{N} \sum_{i=1}^N S_i \frac{\sum_{t=0}^T \mathbb{I}(a'_t \in \mathcal{A}_{change})}{\sum_{t=0}^T \mathbb{I}(a'_t \in \mathcal{A}_{all})}$, where $\mathbb{I}(\cdot)$ is the indicator function, a'_t is the action taken by the agent at timestamp t of the i th episode, $\mathcal{A}_{all}$ is the available actions, and $\mathcal{A}_{change}$ indicates the actions that can change the agent's locations. We evaluate the performance on all trajectories (ALL) and those with an optimal path length of at least 5 ($L \geq 5$).

4.3. Implementation details

In the object-goal navigation framework, a reward function is designed to minimize the trajectory length required for locating the target object. Following prior studies (Du et al., 2020, 2021; Xu et al., 2024; Zhang et al., 2021), the agent receives a substantial positive reward of 5 upon successfully reaching the target object within a maximum number of steps. Conversely, if the agent fails to find the target object within the allotted steps, it incurs a small negative reward of -0.01 for each step taken. At the beginning of each episode, the agent is initially placed at a random state within a random scene and tasked with finding a random target within a maximum number of steps. In our experiments, we use the Adam optimizer (Kingma & Ba, 2014) to update the network parameters, using a learning rate of 10^{-4} . We conduct a total of 6 million episodes to train our methods. During the evaluation phase, we execute 250 episodes for each scene type, resulting in a total of 1000 episodes. The method that achieves the highest success rate will be evaluated on the test scenes, and its performance will be reported as the final results. Regarding the Faster R-CNN training, we refer to Du et al. (2020) and fine-tune it using 50% of the training data from the environments. We compare our method with the following methods, as shown in Table 1:

Random. The agent selects actions from the action space based on a uniform probability distribution.

Baseline. Zhu et al. (2017) introduces the first framework to use the A3C algorithm to navigate the agent.

Scene Prior (SP). Yang et al. (2018) integrates semantic and functional priors to address the task of object-goal navigation using the RL algorithm.

Self-Adaptive Visual Navigation (SAVN). Wortsman et al. (2019) introduces the ML to enable the agent to adapt to unseen environments and adopt Glove embedding to represent target objects.

Context Grid (CG). Druon et al. (2020) proposes an intermediate representation called context grid to navigate the agent to find the target object.

Object Relation Graph (ORG). Du et al. (2020) proposes an object relation graph to learn co-occurrence relationships among object classes across various environments and adopt a one-hot target vector to represent target objects.

Object Relation Graph and Tentative Policy (ORG + TPN). Du et al. (2020) proposes the object relation graph mentioned above and a memory-augmented tentative policy. This policy offers explicit action instructions during failure steps to update the navigation network.

Memory-utilized Joint hierarchical Object Learning for Navigation in Indoor Rooms (MJOLNIR). Pal et al. (2021) proposes a hierarchical object relationship learning approach to the object-goal navigation problem by understanding the role of semantic context.

Visual Transformer Network (VTNet). Du et al. (2021) introduces VTNet, a Visual Transformer for learning informative visual representations in navigation.

Hierarchical Object-to-Zone Graph (HOZ). Zhang et al. (2021) proposes the hierarchical object-to-zone graph that captures the prior knowledge about objects in typical zones.

History inspired Navigation Policy Learning (HiNL). Du et al. (2023) proposes a history-inspired navigation policy framework to effectively estimate navigation states by utilizing historical states.

Aligning Knowledge Graph with Visual Perception (AKGVP). Xu et al. (2024) proposes to align knowledge graphs with visual perception by leveraging visual-language pre-training.

Besides, recent works such as Dual Adaptive Thinking (DAT) (Dang et al., 2023b) and Multiple Thinking (MT) (Dang et al., 2023a) adopt DETR as the object detector to enhance object detection accuracy and introduce coordinate features as additional inputs to provide spatial cues. For a fair comparison, we also employ DETR to construct our COG and HOG. However, following the task definition in Section 3.1, where the agent perceives only RGB observations, we compare our method with the RGB-only versions of these approaches without coordinate inputs. In particular:

Dual Adaptive Thinking (DAT). Dang et al. (2023b) proposes a dual adaptive thinking method that flexibly adjusts the thinking strategies in different navigation stages.

Table 2

Comparison results in the AI2-Thor environment. We evaluate the SR, SPL, and SAE. $L \geq 5$ denotes that the optimal path is larger than 5 steps. (I) and (II) indicate the use of Faster R-CNN and DETR as the object detectors, respectively.

ID	Method	ALL			$L \geq 5$		
		SR↑(%)	SPL↑(%)	SAE↑(%)	SR↑(%)	SPL↑(%)	SAE↑(%)
I	Random	3.56	1.73	0.41	0.27	0.07	0.06
	Baseline (Zhu et al., 2017)	57.35	33.78	19.02	45.77	30.65	20.04
	SP (Yang et al., 2018)	62.16	37.01	23.39	50.86	34.17	24.35
	SAVN (Wortsman et al., 2019)	63.32	37.62	21.97	52.38	35.31	24.64
	CG (Druon et al., 2020)	58.53	19.14	–	–	–	–
	ORG (Du et al., 2020)	66.38	38.42	25.36	55.55	36.26	27.53
	ORG + TPN (Du et al., 2020)	67.31	39.53	23.07	57.41	38.27	26.37
	MJOLNIR (Pal et al., 2021)	65.30	21.10	–	50.00	20.90	–
	VTNet (Du et al., 2021)	70.10	39.60	27.02	61.70	39.90	29.72
	HOZ (Zhang et al., 2021)	70.62	40.02	27.97	62.75	39.24	30.14
	HiNL (Du et al., 2023)	71.20	38.90	–	64.10	37.00	–
	AKGVP (Xu et al., 2024)	73.63	40.66	28.57	63.51	39.63	31.37
	Ours	74.13	42.11	30.67	66.76	42.43	33.19
II	DAT (Dang et al., 2023b)	76.10	43.75	29.97	68.34	43.58	31.86
	MT (Dang et al., 2023a)	77.32	44.71	30.43	69.15	44.53	32.00
	Ours	78.10	45.32	31.23	70.99	46.44	34.11

Table 3

Comparison results in the RoboThor environment. We evaluate the SR, SPL, and SAE. $L \geq 5$ denotes that the optimal path is larger than 5 steps. (I) and (II) indicate the use of Faster R-CNN and DETR as the object detectors, respectively.

ID	Method	ALL			$L \geq 5$		
		SR↑(%)	SPL↑(%)	SAE↑(%)	SR↑(%)	SPL↑(%)	SAE↑(%)
I	Random	0.00	0.00	0.00	0.00	0.00	0.00
	Baseline (Zhu et al., 2017)	26.41	16.61	13.15	17.42	12.23	10.94
	SP (Yang et al., 2018)	28.04	17.63	14.23	21.66	15.14	13.27
	SAVN (Wortsman et al., 2019)	28.42	17.82	13.91	22.13	15.34	13.01
	ORG (Du et al., 2020)	29.61	19.23	14.72	22.53	15.73	13.82
	ORG + TPN (Du et al., 2020)	30.01	20.51	14.52	22.25	16.64	13.83
	VTNet (Du et al., 2021)	31.62	19.63	15.89	23.48	16.89	14.87
	HOZ (Zhang et al., 2021)	32.27	20.48	17.18	24.83	17.18	15.62
	AKGVP (Xu et al., 2024)	33.53	21.68	18.47	25.41	17.74	16.68
	Ours	35.32	22.95	19.04	26.35	18.24	17.41
II	DAT (Dang et al., 2023b)	37.87	23.52	19.64	31.62	19.96	18.27
	MT (Dang et al., 2023a)	38.70	24.34	20.45	32.73	20.54	18.95
	Ours	39.10	25.51	20.93	33.19	21.26	19.59

Multiple Thinking (MT). Dang et al. (2023a) proposes a multiple thinking model that leverages distinct thinking and meta-ability (MA) reward mechanism.

4.4. Results

To comprehensively demonstrate the generalization ability of our proposed method, we conduct both quantitative and qualitative analysis, comparing it with existing navigation methods. These findings provide a comprehensive evaluation and underscore the advantages of our proposed method when navigating in unfamiliar environments.

4.4.1. Quantitative results

As shown in Tables 2 (I) and 3 (I), our proposed method exhibits the best performance in both AI2-Thor and RoboThor environments compared to existing methods using Faster R-CNN as the object detector. In the AI2-Thor environment, our method achieves the highest SR of 74.13%, the highest SPL of 42.11%, and the highest SAE of 30.67% on all trajectories (ALL), surpassing HOZ and AKGVP with gains of 3.51%/2.09%/2.70% and 0.50%/1.45%/2.10% in SR/SPL/SAE. Besides, our method achieves the highest SR of 66.76%, the highest SPL of 42.43%, and the highest SAE of 33.19% on trajectories with an optimal path length of at least 5 ($L \geq 5$), surpassing HOZ and AKGVP with the gains of 4.01%/3.19%/3.05% and 3.25%/2.80%/1.82% in SR/SPL/SAE. In the RoboThor environment, our method achieves the highest SR of 35.32%, the highest SPL of 22.95%, and the highest SAE of

19.04% on all trajectories (ALL), surpassing HOZ and AKGVP with gains of 3.05%/2.47%/1.86% and 1.79%/1.27%/0.57% in SR/SPL/SAE. Besides, our method achieves the highest SR of 26.35%, the highest SPL of 18.24%, and the highest SAE of 17.41% on trajectories with an optimal path length of at least 5 ($L \geq 5$), surpassing HOZ and AKGVP with the gains of 1.52%/1.06%/1.79% and 0.94%/0.50%/0.73% in SR/SPL/SAE.

Since SP uses a category-wise object graph extracted from a real-world Visual Genome (VG) database, it could result in a generalization gap when transferring the object graph from an external database to the agent's environment. SP may tend to fail in navigating to the wrong areas in unfamiliar environments or recognize the wrong objects as targets. Our proposed method integrates a dual object graph with more accurate object relations obtained from object detection extracted from the environment, enhancing the efficiency and effectiveness of navigation in unfamiliar environments.

SAVN uses ML to improve navigation policy learning. However, SAVN does not introduce an object graph to help the agent learn about object relationships, leading to few performance improvements. ORG, ORG + TPN, VTNet, HOZ, HiNL, and AKGVP further introduce object graphs and use the GCN to learn graph representations, resulting in more generalization ability enhancement. Our proposed method uses COG and HOG to help the agent learn both real-time and long-term object relationships and uses GAT to focus on valuable relationships, thus improving visual representations. Besides, we use BM to learn task-relevant information, thus guiding the agent to take effective actions. Therefore,

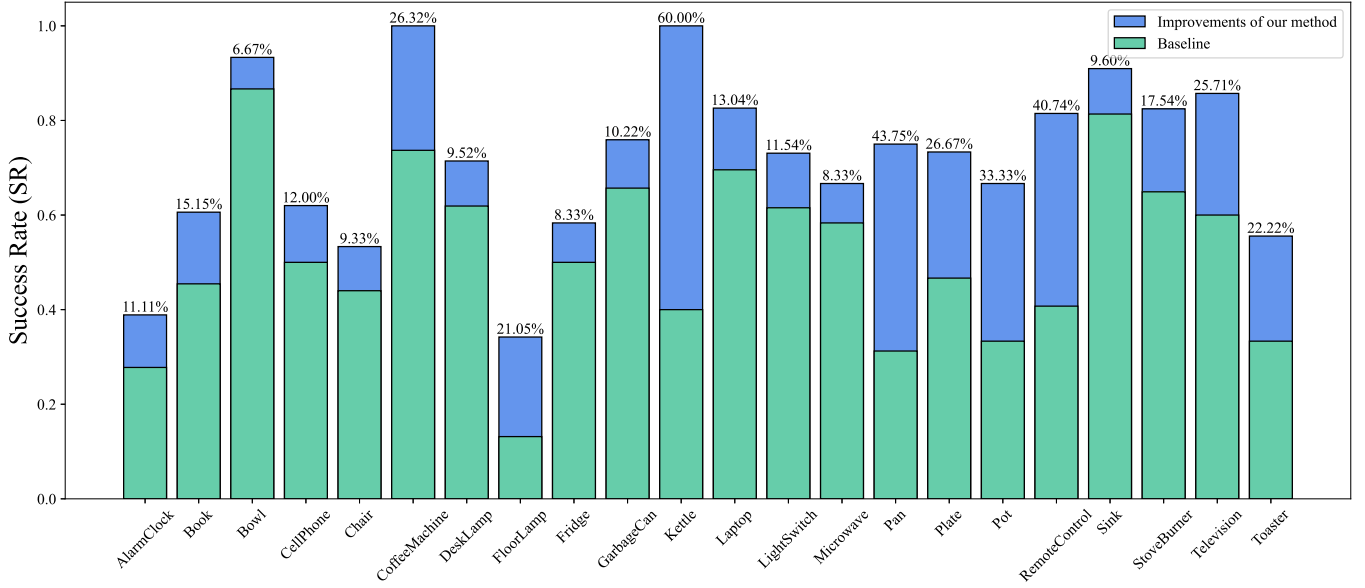


Fig. 4. Object-level improvements of our proposed method over the Baseline in success rate (SR). We choose the 22 target objects to represent, including AlarmClock, Book, Bowl, CellPhone, Chair, CoffeeMachine, DeskLamp, FloorLamp, Fridge, GarbageCan, Kettle, Laptop, LightSwitch, Microwave, Pan, Plate, Pot, RemoteControl, Sink, StoveBurner, Television, Toaster.

our proposed method can achieve the best generalization ability when navigating the agent in unfamiliar environments.

As illustrated in Fig. 4, we compared our proposed method with the Baseline at the object level in the AI2-Thor environment to show our method's improvements in SR. Furthermore, we conduct additional experiments using DETR as the object detector and compare our methods with recent works, including DAT and MT. As shown in Tables 2 (II) and 3 (II), our method consistently achieves superior navigation performance under the same settings. These findings demonstrate that our method significantly improves the effectiveness and efficiency in unfamiliar environments.

4.4.2. Qualitative results

As illustrated in Figs. 5 and 6, we qualitatively compare our proposed method with HOZ and AKGVP in the AI2-Thor and RoboThor environments. In these scenes, the agent is initially placed at random locations where the target object is not visible. HOZ and AKGVP often get stuck in rotations when the target object is not visible and issue *Done* action too early. However, our proposed method enables the agent to understand its surroundings and find a short path to the target object.

With the benefits of DOG and BM, the agent learns task-relevant policy and takes effective actions compared to HOZ and AKGVP. As shown in the first and second rows in Fig. 5, when the agent is tasked with finding a microwave in the kitchen and a television in the living room, HOZ navigates the agent toward the wrong direction and AKGVP takes redundant rotations, while our method can take shortcuts. As shown in the third row, when the agent is tasked with finding an alarm clock in the bedroom, HOZ and AKGVP consistently get stuck and eventually fail within the maximum steps, whereas our proposed method assists the agent in avoiding the bed and locating the alarm clock. As shown in the fourth row, when the agent is tasked with finding a garbage can in the bathroom, HOZ executes *Done* too early, and our method successfully executes *Done* and takes fewer steps compared to AKGVP. Besides, as shown by the first and second rows in Fig. 6, when finding a cell phone and a laptop in large scenes in the RoboThor environment, both HOZ and AKGVP select wrong search areas, whereas our method enables the agent to recognize the potential location of these target objects and successfully find them. As shown in the third and fourth rows, when the

Table 4

The ablation study of different components. We evaluate SR, SPL, and SAE. $L \geq 5$ denotes that the optimal path is larger than 5 steps.

COG	HOG	BM	ALL			$L \geq 5$		
			SR↑(%)	SPL↑(%)	SAE↑(%)	SR↑(%)	SPL↑(%)	SAE↑(%)
✓			71.38	39.19	28.23	62.53	38.84	29.77
	✓		70.16	40.29	27.65	62.38	39.91	30.13
✓		✓	72.70	40.98	27.83	64.50	40.88	30.14
	✓		72.19	40.68	26.76	62.83	40.91	30.12
✓	✓	✓	74.13	42.11	30.67	66.76	42.43	33.19

agent is tasked to find a desk lamp and a book, our method can take fewer steps.

Furthermore, Figs. 7 and 8 show extra visualization examples to demonstrate the superior navigation performance of our method. In these figures, the first-person views of the agent are given, and the agent is tasked with different objects in four kinds of unfamiliar scenes. As shown in Fig. 7, the first column is the kitchen, and the target object is a bowl. The second column is the living room, and the target object is a remote control. The third column is the bedroom, and the target object is a book. The fourth column is the bathroom, and the target object is a light switch. Besides, Fig. 8 showcases examples in the RoboThor environment, where the agent seeks an alarm clock (first column), a garbage can (second column), a plate (third column), and a television (fourth column). Our method has good generalization ability in various unfamiliar scenes, even in some big scenes, such as the living room. Our method can take valid steps when no target objects appear in observation. Besides, for the small objects, such as the remote control or the alarm clock, our method still can navigate the agent to the target object in fewer steps.

4.5. Ablation study

4.5.1. Ablation studies of different components

We conduct the ablation study to evaluate the impact of DOG (i.e., COG and HOG) and BM in our method. The ablation study shown in Table 4 demonstrates the efficacy of each component. We observe that

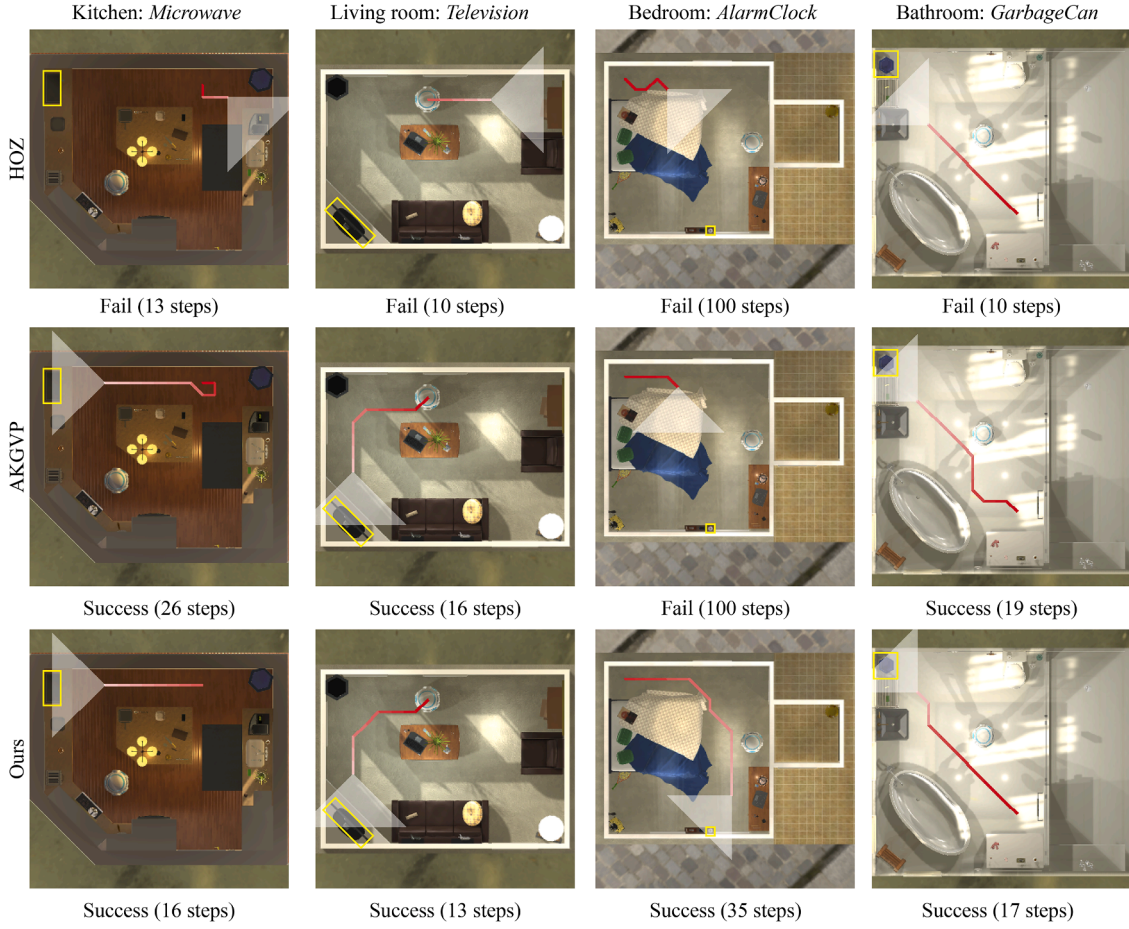


Fig. 5. Visualization of the agent's trajectories in unfamiliar scenes in the AI2-Thor environment. The red lines depict the agent's trajectories, while the yellow boxes indicate the locations of the target objects. Besides, we represent the terminal visibility cone in white.

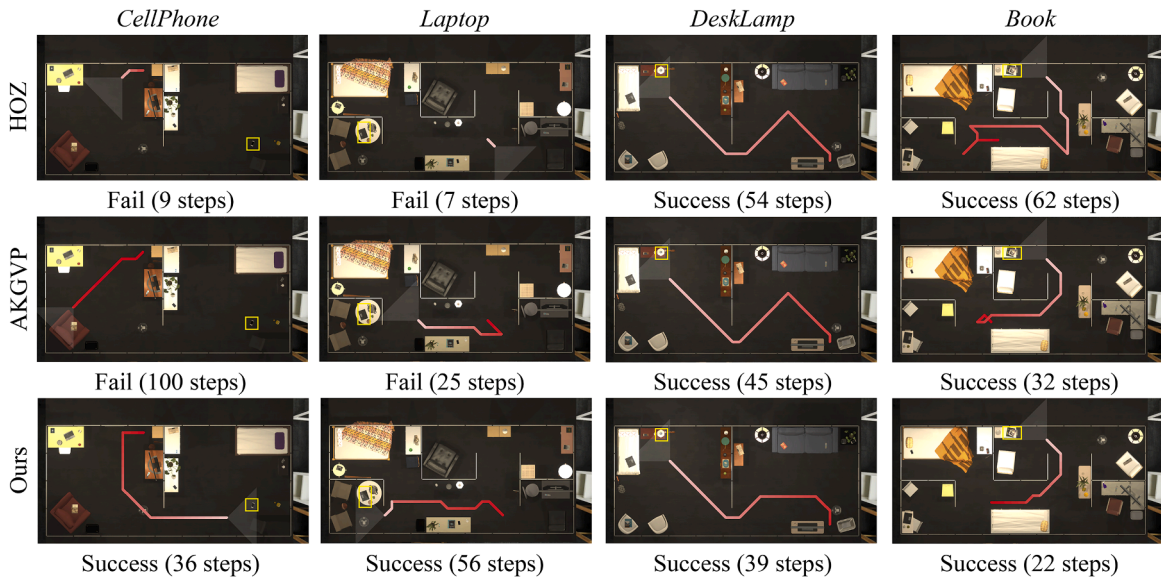


Fig. 6. Visualization of the agent's trajectories in unfamiliar scenes in the RoboThor environment. The red lines depict the agent's trajectories, while the yellow boxes indicate the locations of the target objects. Besides, we represent the terminal visibility cone in white.

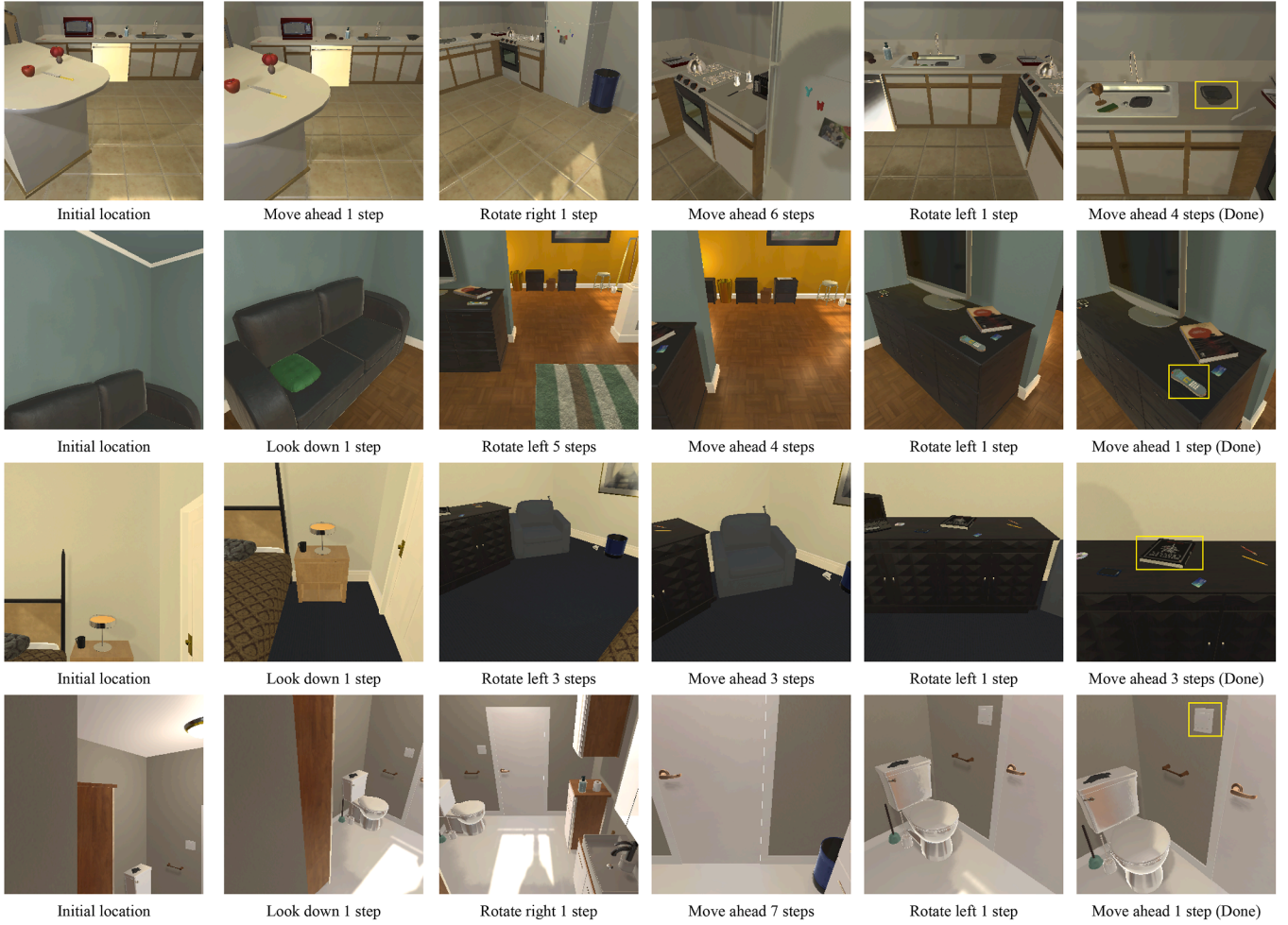


Fig. 7. Visualization of the agent's first-person view in unfamiliar scenes in the AI2-Thor environment. The agent is initially placed at the location where the target object is not visible. The target objects found by the agent are marked by yellow boxes.

Table 5
Computational cost of DOG with different numbers of objects.

Objects	FLOPs (M)	Params (M)	Model Size (M)
10	12.520	9.751	37.349
20	13.581	9.753	37.355
50	16.765	9.756	37.377
100	22.072	9.763	37.430

both COG and HOG can considerably enhance performance in terms of SR, SPL, and SAE. When combining COG and HOG as DOG, it outperforms either of the two. Besides, BM achieves considerable performance improvements by learning effective navigation policy. Furthermore, integrating DOG and BM leads to significant improvements on SR, SPL, and SAE, particularly on long-distance trajectories ($L \geq 5$), demonstrating the effectiveness and efficiency of our proposed method in unfamiliar environments.

4.5.2. Scalability analysis of DOG

We analyze the computational cost of DOG with varying numbers of objects for indoor environments. As shown in Table 5, while the total number of parameters remains nearly constant (~ 9.75 M), FLOPs increases moderately as the number of nodes grows from 10 to 100 (12.5M to 22.1M), indicating that the model is scalable for the tested object counts. The model size in memory also increases slightly with node count, reflecting the additional storage required for object features. For larger graphs containing more objects, computational cost

Table 6
Comparison of different representation learning methods.

Method	ALL			$L \geq 5$		
	SR $\uparrow$ (%)	SPL $\uparrow$ (%)	SAE $\uparrow$ (%)	SR $\uparrow$ (%)	SPL $\uparrow$ (%)	SAE $\uparrow$ (%)
Baseline	57.35	33.78	19.02	45.77	30.65	20.04
CPC (van den et al., 2018)	64.87	37.26	23.14	52.59	36.18	24.73
Proto-RL (Yarats et al., 2021)	68.23	39.12	25.41	57.28	38.76	27.85
BM (Ours)	72.19	40.68	26.76	62.83	40.91	30.12

could increase further due to dense GAT attention. To mitigate this, sparsification strategies such as limiting attention to the top-k most relevant neighbors or using a sparse adjacency matrix can be applied, reducing both runtime and memory usage without affecting performance. Besides, training memory is higher than inference due to gradient storage, but the trend with node count remains similar, and sparsification can also help during training.

4.5.3. Effectiveness of BM

To evaluate the effectiveness of the proposed BM, we project learned state representations into a 2D space using t-SNE for qualitative comparison. As illustrated in Fig. 9, state representations learned without BM exhibit disordered and irregular transitions, indicating that the agent's internal states are unstable and easily affected by local visual variations. In contrast, BM yields a smooth and coherent distribution that

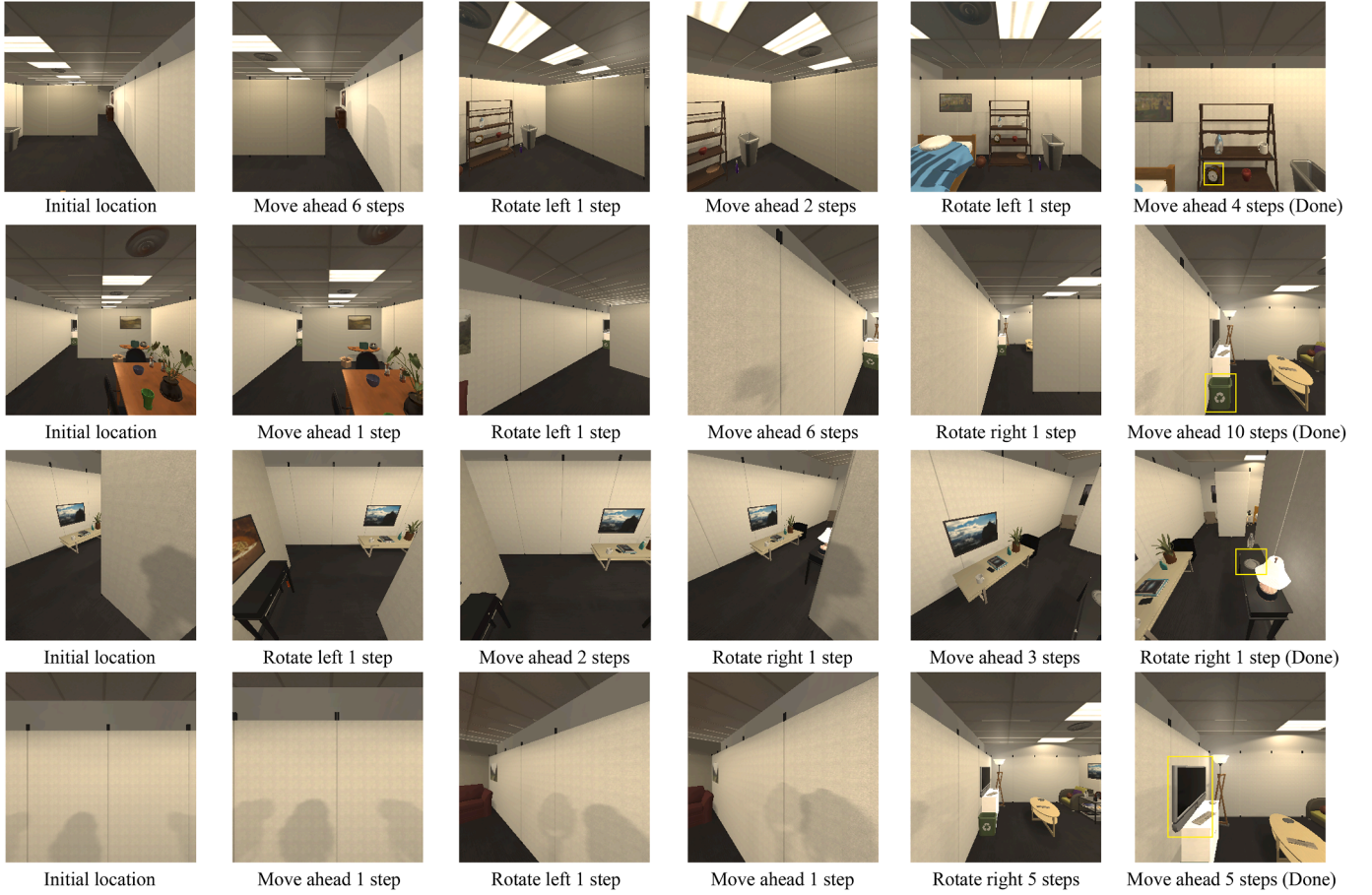


Fig. 8. Visualization of the agent's first-person view in unfamiliar scenes in the RoboThor environment. The agent is initially placed at the location where the target object is not visible. The target objects found by the agent are marked by yellow boxes.

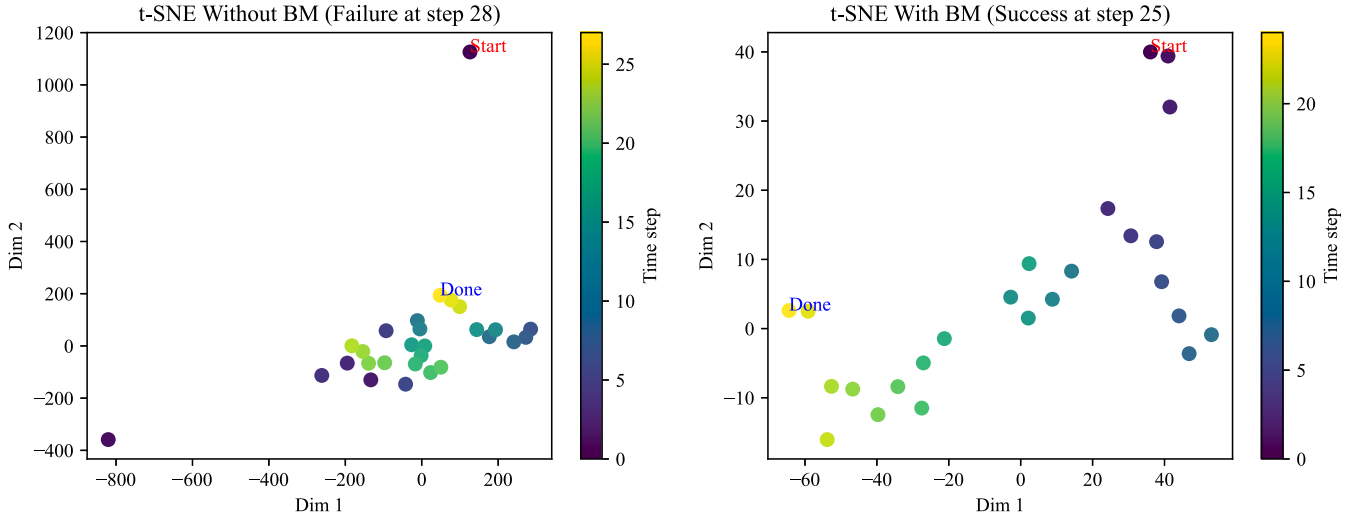


Fig. 9. t-SNE visualization of state representations over the same navigation task for finding a television. Each point corresponds to a state representation at a specific time step, and the gradient color indicates the time step. Without BM (left), state representations are disordered and irregular, while with BM (right), state representations form a smoother and more coherent distribution.

aligns with the navigation process, suggesting that BM encourages behaviorally consistent and task-relevant representations. This stabilized structure enables more reliable decision-making, leading to improved navigation performance in unfamiliar environments.

Furthermore, as shown in Table 6, we further compare BM with two representation learning methods, including CPC (van den et al., 2018) and Proto-RL (Yarats et al., 2021). While all these methods improve

navigation performance over the Baseline, BM consistently achieves the best results across SR, SPL, and SAE. CPC provides limited gains because its contrastive objective predicts future observations without modeling reward or transitions, limiting its ability to learn task-relevant information. Proto-RL performs better by leveraging prototypical state clustering. However, without explicit guidance from rewards, its representations may still encode task-irrelevant information. In contrast, BM

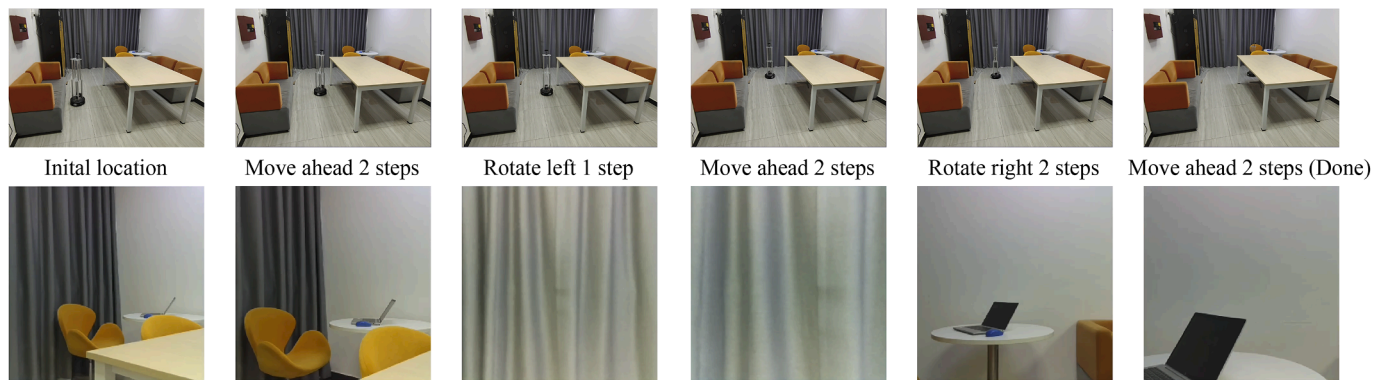


Fig. 10. Visualization of the robot's third-person view and first-person view. The target object is a laptop.

explicitly aligns states with similar immediate reward and distribution over the next states, leading to task-relevant representations that generalize more effectively in unfamiliar environments.

4.6. Real-world deployment

We develop a real-world system to evaluate the transferability of our proposed method in a real-world test environment. We deploy our method trained in the AI2-Thor environment on a Turtlebot4-Lite wheeled robot equipped with an OAK-D-Lite camera mounted at a height of 1.0 m above the ground. The navigation relies solely on RGB camera frames, and Faster R-CNN pre-trained on the COCO dataset is used for object detection. All algorithms run in real-time on a laptop with an R7-8745H CPU and an NVIDIA GeForce RTX 4050 GPU. A sample navigation trajectory of successfully finding a laptop is visualized in Fig. 10. The deployment demonstrates the effectiveness of our method in transferring navigation capabilities from simulation to real-world environments.

5. Conclusion

In this paper, we propose an object-goal navigation method using a dual object graph and bisimulation metric. Benefiting from our proposed current object graph and historical object graph, the agent can localize the target objects effectively and efficiently in unfamiliar environments. Furthermore, our proposed bisimulation metric learns robust task-relevant representations against distractions, thus guiding the agent to take effective actions. Experiments demonstrate that our method significantly improves the effectiveness and efficiency of navigation in unfamiliar environments. We further deploy our method on a Turtlebot4-Lite wheeled robot to demonstrate its transferability in a real-world environment. Future work may focus on extending our method to other navigation tasks (e.g., point-goal navigation, image-goal navigation, and vision-language navigation). Moreover, further exploration is necessary on how to narrow the sim-to-real gap across diverse real-world scenarios.

CRedit authorship contribution statement

Yiyue Meng: Writing – review & editing, Writing – original draft, Methodology; **Chi Guo:** Writing – review & editing, Conceptualization; **Aolin Li:** Conceptualization; **Kang Zhou:** Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work is supported in part by the Major Science and Technology Project of Hubei Province under Grant 2022AAA009, and the Open Fund of Hubei Luojia Laboratory.

References

- Castro, P. S. (2020). Scalable methods for computing state similarity in deterministic markov decision processes. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 10069–10076). (vol. 34).
- Dang, R., Chen, L., Wang, L., He, Z., Liu, C., & Chen, Q. (2023a). Multiple thinking achieving meta-ability decoupling for object navigation. In *International conference on machine learning* (pp. 6855–6872). PMLR.
- Dang, R., Wang, L., He, Z., Su, S., Tang, J., Liu, C., & Chen, Q. (2023b). Search for or navigate to? Dual adaptive thinking for object navigation. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 8250–8259).
- Deitke, M., Han, W., Herrasti, A., Kembhavi, A., Kolve, E., Mottaghi, R., Salvador, J., Schwenk, D., VanderBilt, E., Wallingford, M. et al. (2020). Robothor: An open simulation-to-real embodied AI platform. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 3164–3174).
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition* (pp. 248–255). IEEE.
- Dissanayake, M. G., Newman, P., Clark, S., Durrant-Whyte, H. F., & Csorba, M. (2001). A solution to the simultaneous localization and map building (SLAM) problem. *IEEE Transactions on Robotics and Automation*, 17 (3), 229–241.
- Druon, R., Yoshiyasu, Y., Kanazaki, A., & Watt, A. (2020). Visual object search by learning spatial context. *IEEE Robotics and Automation Letters*, 5 (2), 1279–1286.
- Du, H., Li, L., Huang, Z., & Yu, X. (2023). Object-goal visual navigation via effective exploration of relations among historical navigation states. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 2563–2573).
- Du, H., Yu, X., & Zheng, L. (2020). Learning object relation graph and tentative policy for visual navigation. In *Computer vision—ECCV 2020: 16th european conference, glasgow, UK, august 23–28, 2020, proceedings, part VII 16* (pp. 19–34). Springer.
- Du, H., Yu, X., & Zheng, L. (2021). VTNet: Visual transformer network for object goal navigation. *arXiv:2105.09447*
- Ferns, N., Panangaden, P., & Precup, D. (2011). Bisimulation metrics for continuous markov decision processes. *SIAM Journal on Computing*, 40 (6), 1662–1714.
- Ferns, N., & Precup, D. (2014). Bisimulation metrics are optimal value functions. In *Uai* (pp. 210–219).
- Givan, R., Dean, T., & Greig, M. (2003). Equivalence notions and model minimization in markov decision processes. *Artificial Intelligence*, 147(1–2), 163–223.
- Grill, J.-B., Strub, F., Altché, F., Tallec, C., Richemond, P., Buchatskaya, E., Doersch, C., Avila Pires, B., Guo, Z., Gheshlaghi Azar, M. et al. (2020). Bootstrap your own latent: a new approach to self-supervised learning. *Advances in Neural Information Processing Systems*, 33, 21271–21284.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770–778).
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9 (8), 1735–1780.
- Jonschkowski, R., & Brock, O. (2015). Learning state representations with robotic priors. *Autonomous Robots*, 39, 407–428.
- Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv:1412.6980*
- Kipf, T. N., & Welling, M. (2016). Semi-supervised classification with graph convolutional networks. *arXiv:1609.02907*
- Kolve, E., Mottaghi, R., Han, W., VanderBilt, E., Weihs, L., Herrasti, A., Deitke, M., Ehsani, K., Gordon, D., Zhu, Y. et al. (2017). AI2-thor: An interactive 3D environment for visual AI. *arXiv:1712.05474*

- Krishna, R., Zhu, Y., Groth, O., Johnson, J., Hata, K., Kravitz, J., Chen, S., Kalantidis, Y., Li, L.-J., Shamma, D. A. et al. (2017). Visual genome: Connecting language and vision using crowdsourced dense image annotations. *International Journal of Computer Vision*, 123, 32–73.
- Larsen, K. G., & Skou, A. (1989). Bisimulation through probabilistic testing (preliminary report). In *Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on principles of programming languages* (pp. 344–352).
- Li, B., Han, J., Cheng, Y., Tan, C., Qi, P., Zhang, J., & Li, X. (2022). Object goal navigation in embodied AI: A survey. In *Proceedings of the 2022 4th international conference on video, signal and image processing* (pp. 87–92).
- Li, L., Walsh, T. J., & Littman, M. L. (2006). Towards a unified theory of state abstraction for MDPs. *AI&M*, 1 (2), 3.
- Mayo, B., Hazan, T., & Tal, A. (2021). Visual navigation with spatial attention. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 16898–16907).
- Mazouze, B., Tachet des Combes, R., Doan, T. L., Bachman, P., & Hjelm, R. D. (2020). Deep reinforcement and infomax learning. *Advances in Neural Information Processing Systems*, 33, 3686–3698.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., & Kavukcuoglu, K. (2016). Asynchronous methods for deep reinforcement learning. In *International conference on machine learning* (pp. 1928–1937). PMLR.
- Mousavian, A., Toshev, A., Fišer, M., Košecká, J., Wahid, A., & Davidson, J. (2019). Visual representations for semantic target driven navigation. In *2019 international conference on robotics and automation (ICRA)* (pp. 8846–8852). IEEE.
- Oriolo, G., Vendittelli, M., & Ulivi, G. (1995). On-line map building and navigation for autonomous mobile robots. In *Proceedings of 1995 IEEE international conference on robotics and automation* (pp. 2900–2906). IEEE (vol. 3).
- Pal, A., Qiu, Y., & Christensen, H. (2021). Learning hierarchical relationships for object-goal navigation. In *Conference on robot learning* (pp. 517–528). PMLR.
- Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in Neural Information Processing Systems*, 28, 1137–1149.
- Sun, J., Wu, J., Ji, Z., & Lai, Y.-K. (2024). A survey of object goal navigation. *IEEE Transactions on Automation Science and Engineering*, 22, 2292–2308.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2017). Graph attention networks. arXiv:1710.10903
- Villani, C. et al. (2009). Optimal transport: old and new (vol. 338). Springer.
- van den, O. A., Li, Y., & Vinyals, O. (2018). Representation learning with contrastive predictive coding. arXiv:1807.03748
- Wang, Y., Yang, M., Dong, R., Sun, B., Liu, F. et al. (2023). Efficient potential-based exploration in reinforcement learning using inverse dynamic bisimulation metric. *Advances in Neural Information Processing Systems*, 36, 38786–38797.
- Wortsman, M., Ehsani, K., Rastegari, M., Farhadi, A., & Mottaghi, R. (2019). Learning to learn how to learn: Self-adaptive visual navigation using meta-learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 6750–6759).
- Xu, N., Wang, W., Yang, R., Qin, M., Lin, Z., Song, W., Zhang, C., Gu, J., & Li, C. (2024). Aligning knowledge graph with visual perception for object-goal navigation. arXiv:2402.18892
- Yang, W., Wang, X., Farhadi, A., Gupta, A., & Mottaghi, R. (2018). Visual semantic navigation using scene priors. arXiv:1810.06543
- Yarats, D., Fergus, R., Lazaric, A., & Pinto, L. (2021). Reinforcement learning with prototypical representations. In *International conference on machine learning* (pp. 11920–11931). PMLR.
- Zhang, A., McAllister, R., Calandra, R., Gal, Y., & Levine, S. (2020). Learning invariant representations for reinforcement learning without reconstruction. arXiv:2006.10742
- Zhang, L., Wang, Z., Li, X., & Li, Y.-H. (2025). Revisiting bisimulation metric for robust representations in reinforcement learning. arXiv:2507.18519
- Zhang, S., Song, X., Bai, Y., Li, W., Chu, Y., & Jiang, S. (2021). Hierarchical object-to-zone graph for object navigation. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 15130–15140).
- Zhang, T., Hu, X., Xiao, J., & Zhang, G. (2022). A survey of visual navigation: From geometry to embodied AI. *Engineering Applications of Artificial Intelligence*, 114, 105036.
- Zhou, K., Guo, C., Guo, W., & Zhang, H. (2023). Learning heterogeneous relation graph and value regularization policy for visual navigation. *IEEE Transactions on Neural Networks and Learning Systems*, 35, 16901–16915.
- Zhu, Y., Mottaghi, R., Kolve, E., Lim, J. J., Gupta, A., Fei-Fei, L., & Farhadi, A. (2017). Target-driven visual navigation in indoor scenes using deep reinforcement learning. In *2017 IEEE international conference on robotics and automation (ICRA)* (pp. 3357–3364). IEEE.